Grafos: Conectando o Mundo Digital

Mário Leite

O que é um Grafo?

Um **grafo** é uma estrutura de dados composta por dois elementos fundamentais: **vértices** (ou nós) e **arestas** (ou arcos) que os conectam. Matematicamente, um grafo G é definido como G = (V, A), onde V é um conjunto de vértices e A é um conjunto de arestas.

Pense em um mapa: as cidades são os vértices e as estradas que as ligam são as arestas. Essa simplicidade é justamente o que torna os grafos tão poderosos.

Por que Grafos são Importantes?

Os grafos são **a estrutura mais universal da computação**. Eles modelam relações, conexões e redes — e quase tudo no mundo real pode ser representado como uma rede de conexões.

Diferente de listas e árvores, que possuem hierarquias rígidas, os grafos permitem que **qualquer elemento se conecte a qualquer outro**, com ou sem direção, com ou sem pesos (custos). Essa flexibilidade os torna indispensáveis.

Aplicações Práticas

Redes Sociais: Cada pessoa é um vértice; cada amizade, uma aresta. O Facebook e o LinkedIn usam grafos para sugerir amigos e detectar comunidades.

GPS e Rotas: O Google Maps transforma ruas em arestas e cruzamentos em vértices. Algoritmos como Dijkstra encontram o caminho mais curto.

Internet: Roteadores formam um grafo gigante. Pacotes de dados buscam o caminho mais eficiente.

Inteligência Artificial: Redes neurais são grafos. Sistemas de recomendação operam sobre estruturas grafo.

Biologia: Proteínas interagem em redes; moléculas são grafos onde átomos são vértices e ligações são arestas.

Logística: Companhias aéreas e entregas usam grafos para otimizar rotas e reduzir custos.

Da Teoria à Prática: O Programa "Plota Grafo"

Para tornar esses conceitos concretos, o programa **PlotaGrafo**, codificado em Pytjon, implementa um grafo de cinco vértices usando duas representações clássicas: a **matriz de adjacência** e a **lista de adjacências**. A matriz de adjacência define as conexões com 0 (sem aresta) e 1 (com aresta):

		1	2	3	4	5	
1	1 [	0	1	0	0	1]	→ Vértice 1 conecta-se a 2 e 5
2	2 [	1	0	1	1	1]	→ Vértice 2 conecta-se a 1, 3, 4 e 5
3	3 [	0	1	0	1	0]	→ Vértice 3 conecta-se a 2 e 4
4	4 [	0	1	1	0	1]	→ Vértice 4 conecta-se a 2, 3 e 5
5	5 [	1	1	0	1	0]	→ Vértice 5 conecta-se a 1, 2 e 4
6							

O programa converte essa matriz em uma **lista de adjacências** — estrutura mais eficiente para percorrer vizinhos — e depois plota o grafo graficamente, desenhando vértices como círculos e arestas como linhas. O resultado é uma visualização imediata da rede de conexões.

Esse exemplo ilustra três conceitos fundamentais: a **representação matricial** (compacta, boa para consultas), a **representação por listas** (eficiente para travessias) e a **visualização** (que transforma dados abstratos em conhecimento concreto).

Conclusão

Dominar grafos é dominar a linguagem das conexões. O programa Plota Grafo mostra que, com poucas linhas de código, é possível transformar uma matriz de zeros e uns em uma rede visual e compreensível. **Grafos não são apenas matéria de prova: são a espinha dorsal do mundo conectado em que vivemos.**

Vértice	Lista de Adjacências
1	2, 5
2	1, 3, 4, 5
3	2, 4
4	2, 3, 5
5	1, 2, 4

Matriz Adjacências

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Figura 1a - Exemplo de placas geradas pelo programa "PlotaGrafo"

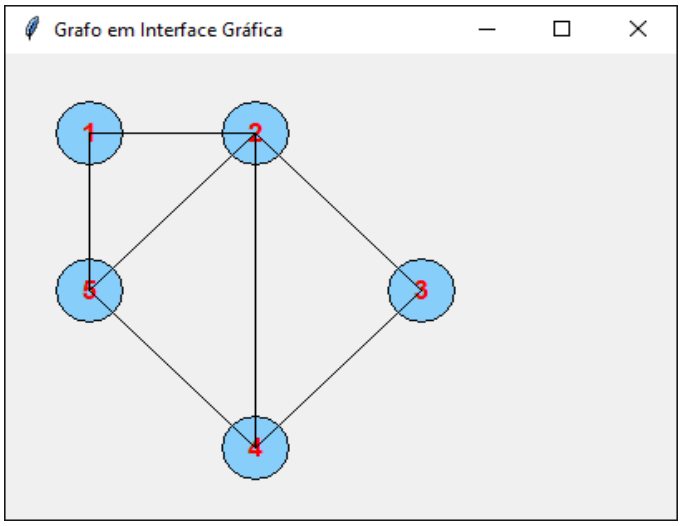


Figura 1.b - Saída do programa "PlotaGrafo"

```
'''
PlotaGrafo.py
-------------------------------------------------------------------------------
Plota um "Grafo" de cinco nós, sujeitos a uma lista e uma matriz de
adjacências.
-------------------------------------------------------------------------------
'''
import tkinter as tk

class ClsGrafoAdjacencia:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.adjacencia = {i+1: [] for i in range(num_vertices)}

    def AdicionarAresta(self, vert, *vizinhos):
        if(vert in self.adjacencia):
            for vizinho in vizinhos:
                if(vizinho not in self.adjacencia[vert]):
                    self.adjacencia[vert].append(vizinho)
                    self.adjacencia[vizinho].append(vert)   #aresta não direcionada

    def ChecarVizinhos(self, v1, v2):
        return v2 in self.adjacencia[v1]

    def LigarVizinho(self, v):
        return self.adjacencia[v]

    def CarregarMatAdjacencia(self, matriz):
        for i in range(self.num_vertices):
            for j in range(self.num_vertices):
                if(matriz[i][j] == 1):
                    self.AdicionarAresta(i+1, j+1)
#-------------------------------------------------------------------------------
class ClsGrafoCanvas(tk.Canvas):
    def __init__(self, master=None):
        super().__init__(master, width=400, height=400)
        self.master = master
        self.pack()

    def DesenharGrafo(self, grafo):
        DicCoordenadas = {
            1: (50, 50),
            2: (150, 50),
            3: (250, 150),
            4: (150, 250),
            5: (50, 150)
        }
        raio = 20
        for vert in range(1, grafo.num_vertices+1):
           x, y = DicCoordenadas[vert]
           self.create_oval(x-raio,y-raio,x+raio,y+raio, fill="#87CEFA")
           self.create_text(x,y,text=str(vert),fill="red",font=("Helvetica",12,"bold"))

        for v1 in range(1, grafo.num_vertices + 1):
            for v2 in grafo.LigarVizinho(v1):
                x1, y1 = DicCoordenadas[v1]
                x2, y2 = DicCoordenadas[v2]
                self.create_line(x1, y1, x2, y2, fill="black")
```

```
#=============================================================================================
#Programa principal
if(__name__ == "__main__"):
    janela = tk.Tk()
    janela.title("Grafo em Interface Gráfica")
    Canvas = ClsGrafoCanvas(master=janela)
    Grafo = ClsGrafoAdjacencia(5)
    LstMatAdjacencia = [
        [0, 1, 0, 0, 1],
        [1, 0, 1, 1, 1],
        [0, 1, 0, 1, 0],
        [0, 1, 1, 0, 1],
        [1, 1, 0, 1, 0]
    ]
    Grafo.CarregarMatAdjacencia(LstMatAdjacencia)
    Canvas.DesenharGrafo(Grafo)
    janela.mainloop()
#Fim do programa "PlotaGrafo" ------------------------------------------------
```